

Mini-Agentic-RAG sobre CineMatch

Pràctica de Laboratori — Tema 3
NoSQL (MongoDB) + IA Generativa

AUTOR

Albert Gil López
albert.gil.lopez@uab.cat
github.com/albertgilopez

CALENDARI

PLAB1 · 21 maig 2026 (2 h)
PLAB2 · 4 juny 2026 (2 h)
Lliurament · 5 juny 2026, 08:30 h

Pràctica de Laboratori · Tema 3

Resum	3
0. Marc conceptual	4
0.1 Embeddings — significat en forma de vector	5
0.2 Vector Search a MongoDB Atlas — què aporta MongoDB	7
0.3 RAG — Retrieval-Augmented Generation	8
0.4 Què aporta MongoDB únicament al RAG (vs Pinecone/Weaviate/...)	9
0.5 Tools, agents i el patró canònic amb MongoDB	10
0.6 Patrons d'arquitectura — workflow vs agent	12
0.7 Tancament ètic — el framework 4D	13
0.8 Recursos per profunditzar (opcionals, gratuïts)	14
1. Context — CineMatch	15
2. El dataset, la vostra còpia, i un avís de correcció crític	16
2.1 El corpus original	17
2.2 Treballem sobre la vostra còpia (no sobre el corpus compartit)	18
2.3 El model d'embedding ha de coincidir EXACTAMENT amb el del corpus	19
3. El repte per nivells (N1 → N4)	20
● N1 — Semantic Search (base, obligatori)	21
● N2 — RAG (retriever + generator)	22
● N3 — Tools + Structured Outputs	23
● N4 — Multi-agent (orquestració)	24
4. Patrons d'arquitectura recomanats	25
5. Lliurables i format	26
6. Restriccions i regles	27
7. Avaluació	28
8. Per on començar (checklist minut 0)	29

RESUM

Construïreu un assistent intel·ligent que respon preguntes en llenguatge natural sobre el catàleg de pel·lícules `sample_mflix`, usant **cerca semàntica** (no `find()` per text exacte) i **generació augmentada per recuperació (RAG)**. El repte té quatre nivells: N1 *Semantic Search*, N2 *RAG*, N3 *Tools + Structured Outputs*, N4 *Multi-agent*. La nota depèn del nivell màxim assolit i **funcionant**; és millor un N2 sòlid que un N4 trencat.

0. MARC CONCEPTUAL

Tots els enllaços van a [Learn.mongodb.com](https://learn.mongodb.com) (gratuït amb el teu correu UAB) i a [Anthropic Academy](https://academy.anthropic.com) (academy.anthropic.com, gratuït).

0.1 Embeddings — significat en forma de vector

Un **embedding** és un array numèric (vector) que **representa el significat** d'una peça de dades (text, imatge, codi). Dades semànticament similars produeixen vectors propers en l'espai: per exemple, dos **cotxes utilitaris** caurien a prop entre ells, i lluny d'un **camió** o d'un **autobús**. Amb 1536 dimensions, el mateix principi però en un espai que no podem visualitzar.

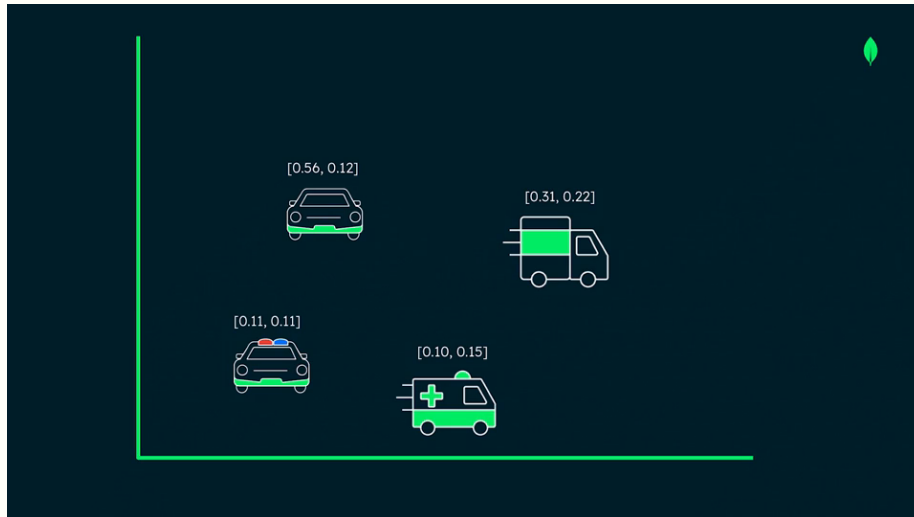


Figura 1. Vehicles representats com a vectors 2D segons les seves característiques: cotxes a la part inferior, camions a la part superior. La proximitat al pla = similitud semàntica. Font: MongoDB University — Using Vector Search for Semantic Search, L01.

La proximitat es mesura amb **cosine similarity** (text, default), **dot product** (vectors normalitzats, més ràpid) o **euclidean** (distància geomètrica). En una cerca semàntica recuperem el **clúster proper** a la query, no fem cerca literal de paraules.



Figura 2. Clústers de cotxes i camions en l'espai vectorial: vectors similars formen agrupacions naturals que el motor de cerca aprofita. Font: MongoDB University — Using Vector Search for Semantic Search, L01.

Dues classes de vectors:

- **Sparse** (TF-IDF, BM25): molts zeros, captura paraules exactes.
- **Dense** (*transformer embeddings*): tots els components amb valor, captura significat — “*rei - home + dona ≈ reina*”.

Regla crítica. El **model** d’embedding de la query **ha de coincidir** amb el del corpus. La dimensió coincident **NO** n’hi ha prou: `ada-002` i `3-small` són tots dos 1536-d però viuen en **espais semàntics diferents** — barrejar-los dona soroll.

 **Vector Search Fundamentals** — MongoDB University

0.2 Vector Search a MongoDB Atlas — què aporta MongoDB

Atlas Vector Search indexa vectors amb **HNSW** (*Hierarchical Navigable Small World*) i fa cerca **aNN** (*approximate nearest neighbor*) — molt més ràpid que el càlcul exacte sobre milions de vectors. El motor entra per la capa superior (poques connexions, salts grans) i baixa fins a la capa 0 (totes les connexions, salts fins). Trobar el veí més proper és **logarítmic**, no lineal.

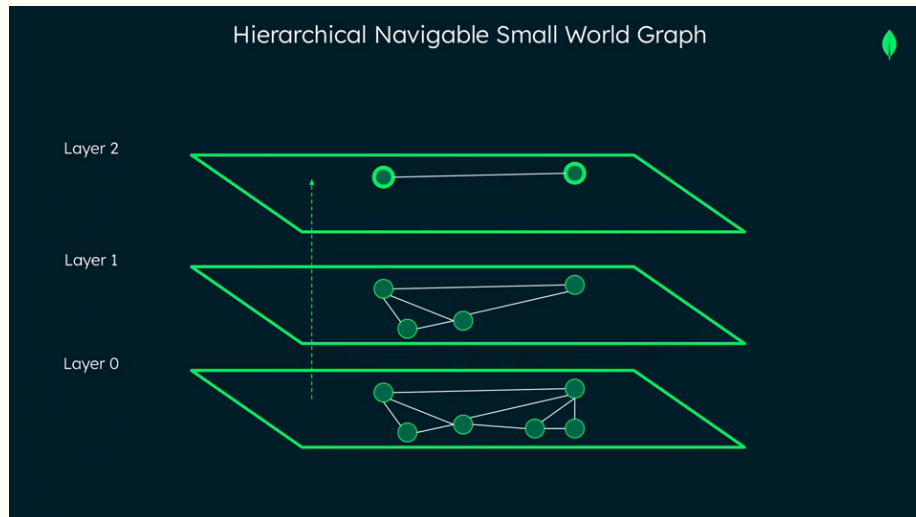


Figura 3. Estructura HNSW: graf de tres capes jeràrquiques (Layer 0 densa, Layer 1 mitjana, Layer 2 escassa) per navegar cap al veí més proper en temps logarítmic. Font: MongoDB University — *Using Vector Search for Semantic Search, L04*.

ANN (aprox., per defecte a Atlas) vs ENN (exacte). ANN ofereix velocitat sobre grans volums amb recall molt alt; ENN garanteix el resultat perfecte però només escala amb pocs vectors. El trade-off d'ANN es controla amb `numCandidates`: més candidats avaluats → més recall, més latència. Per a corpus mitjans (~milers a milions de documents), `numCandidates ≈ 150` sol ser el punt òptim.

L'etapa `$vectorSearch` és part del pipeline d'agregació estàndard:

```
{ "$vectorSearch": {
  "index": "vectorPlotIndex",
  "path": "plot_embedding",
  "queryVector": [0.012, -0.043, ...],
  "numCandidates": 150,    // ANN candidats avaluats (recall ↔ latència)
  "limit": 5 }}           // top-k retornats al client
```

[🔗 Using Vector Search for Semantic Search](#) — MongoDB University

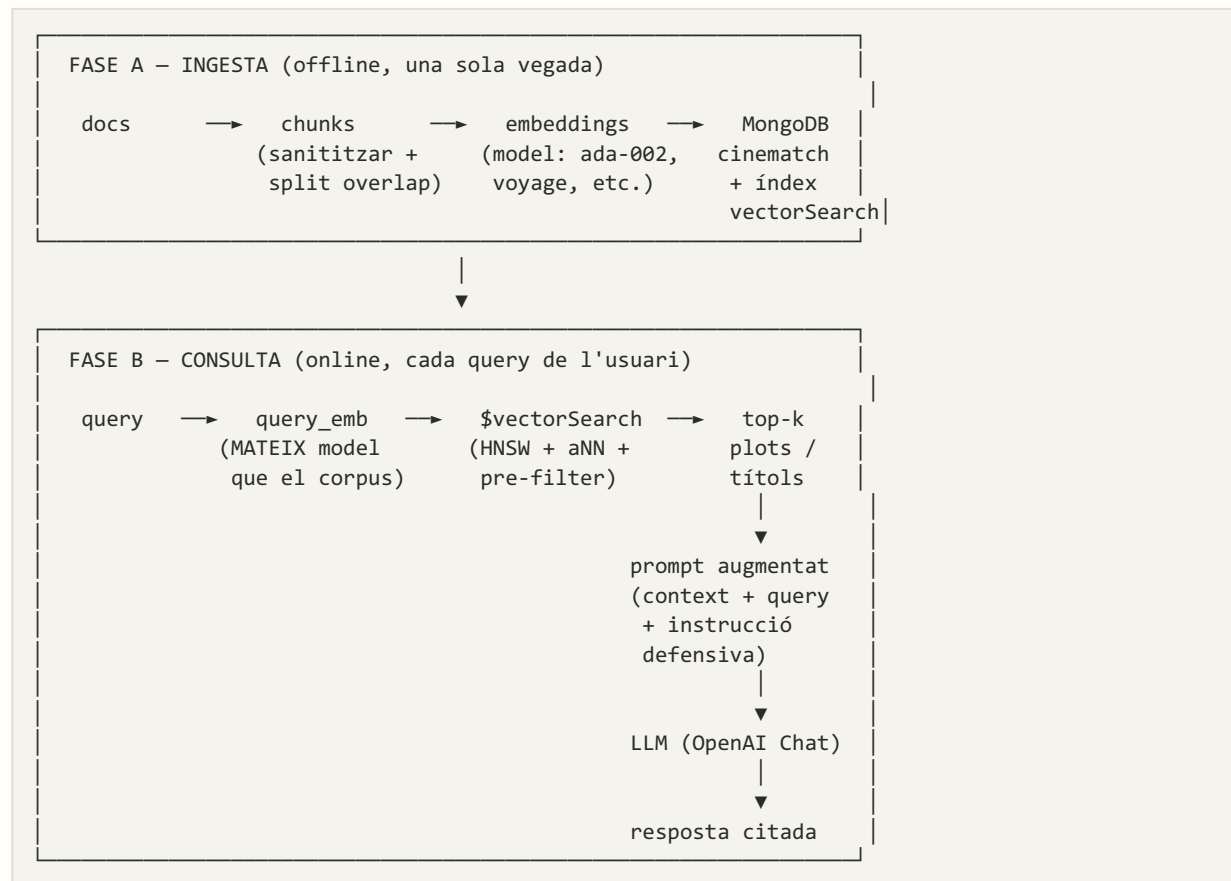
0.3 RAG — Retrieval-Augmented Generation

RAG = combinar un **retriever** (busca documents rellevants) amb un **generator** (LLM que redacta usant aquest context). Soluciona tres límits del LLM pur:

1. **Coneixement congelat** a la data d'entrenament.
2. **Absència de dades privades/internes**.
3. **Al·lucinacions** (inventar respostes plausibles però falses).

El LLM ja no “sap” la resposta — la llegeix del context que li injectes.

Arquitectura RAG en dues fases:



Nota. En aquesta pràctica la FASE A ja està feta. `sample_mflix.embedded_movies` ja porta el camp `plot_embedding` precalculat amb OpenAI `text-embedding-ada-002` (1536-d). Vosaltres només heu d'implementar la **FASE B** (consulta) i la generació de resposta amb LLM. No regeneu el corpus.

Prompt defensiu = última línia contra l'al·lucinació: “si el context no conté la resposta, digues que no ho saps; no inventis”. Sense aquest prompt, no és RAG, és LLM amb context decoratiu.

🔗 **RAG with MongoDB** — MongoDB University

0.4 Què aporta MongoDB únicament al RAG (vs Pinecone/Weaviate/...)

Aquest és l'angle pedagògic del taller: per què val la pena fer RAG en Mongo.

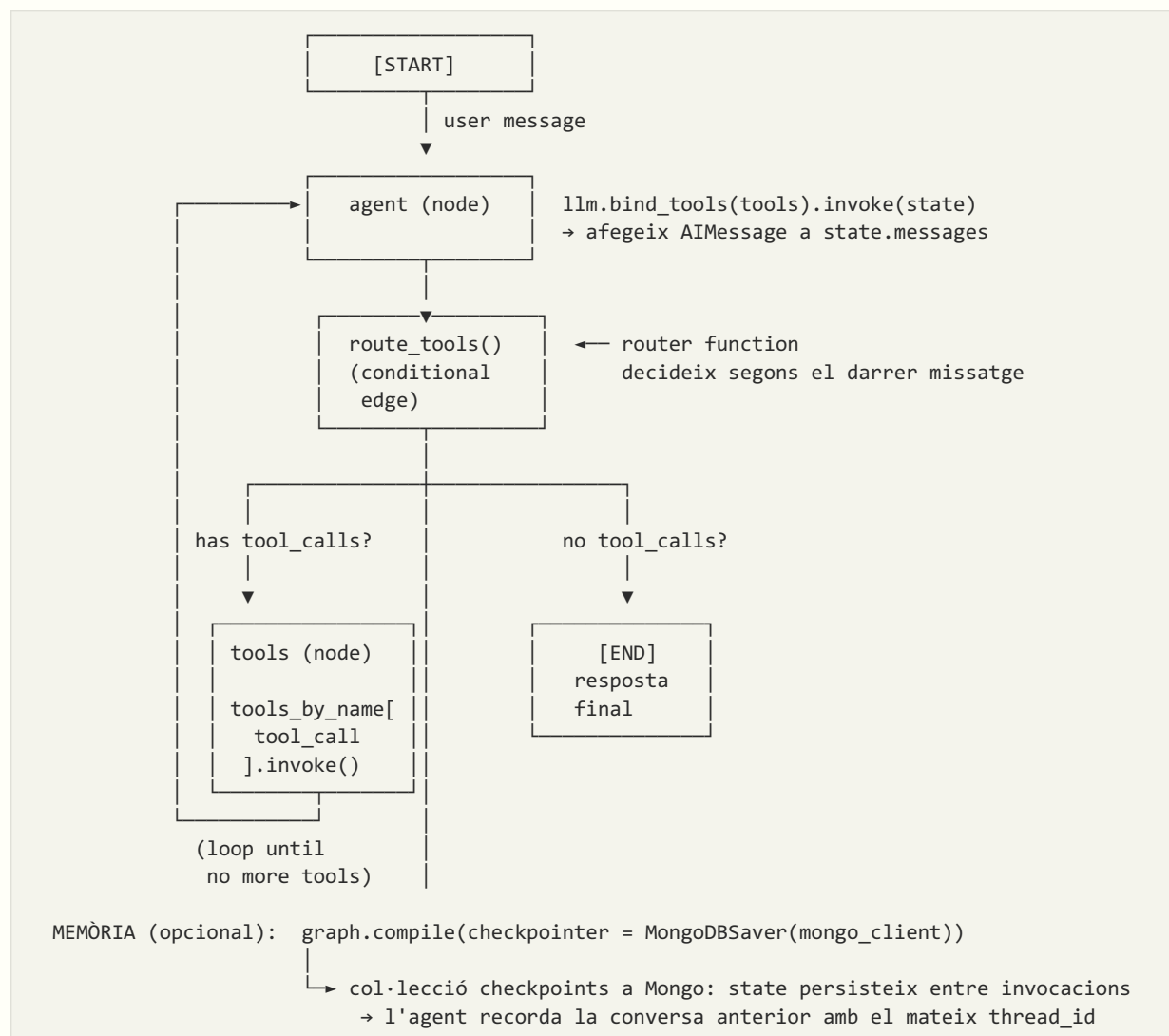
Avantatge	Per què importa
Dades operacionals + vectorials a la mateixa col·lecció	Cap ETL entre BD i vector store extern. Una sola font de veritat.
Pre-filter integrat dins \$vectorSearch (camp declarat filter a l'índex)	Redueix candidats <i>abans</i> del càlcul ANN → més ràpid i més rellevant que un \$match posterior.
Hybrid search natiu (\$vectorSearch + \$search + \$unionWith) amb RRF	Combinar semàntic + lèxic en una sola pipeline d'agregació.
\$vectorSearch és una etapa d'aggregation	Encadenable amb \$match , \$lookup , \$project , \$group . És “una etapa més”, no un sistema apart.
Atlas Triggers per auto-embedding	Quan inseris un document, un trigger pot calcular l'embedding i guardar-lo — ingesta sense codi de sincronització.
MongoDBSaver checkpointer per a LangGraph	MongoDB com a memory store de l'agent (estat conversacional persistit). Vegeu N4.

0.5 Tools, agents i el patró canònic amb MongoDB

Per a N3 i N4, l'estructura canònica que ensenya **MongoDB U “AI Agents with MongoDB”** i que recomanem per al repte:

- **Tools:** funcions Python decorades amb `@tool` (LangChain) o registrades com a function calling (OpenAI). Cada tool té schema, descripció i implementació. L'LLM decideix quina cridar segons la pregunta.
- **State graph (LangGraph):** substitueix el loop manual `while True` per un graf amb **nodes** (`agent` , `tool_node`) i un **router** (`route_tools`) que decideix si hi ha més tool calls o si s'acaba la conversa.
- **Memory amb MongoDBSaver** : la conversa es persisteix a una col·lecció de Mongo entre invocacions → l'agent recorda. Únic de l'ecosistema MongoDB.

Patró canònic — state graph del curs ai-agents-with-mongodb :



Per què aquest patró: és **explícit** (veus el flux), **inspeccionable** (cada node retorna estat consultable), i **persistent** (amb `MongoDBSaver` la memòria viu en la mateixa BD que les teves dades).

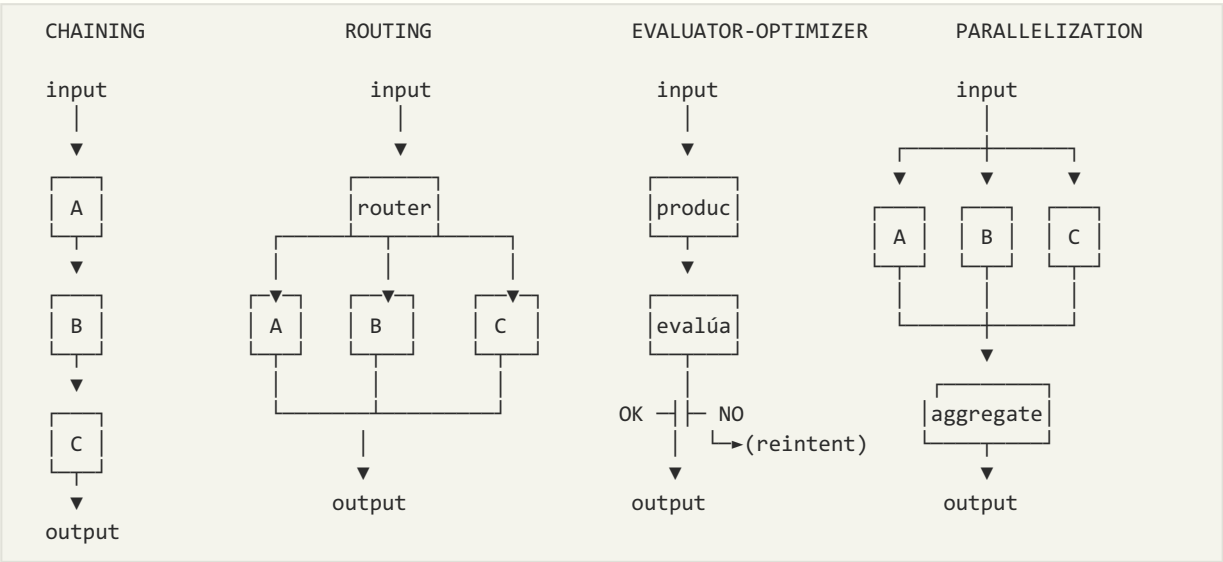
AI Agents with MongoDB — MongoDB University (recomanat per al repte)

Nota de stack: el curs `ai-agents-with-mongodb` usa **Voyage AI** per a embeddings (model preferit de MongoDB des de l'adquisició 2024). Nosaltres mantenim `text-embedding-ada-002` per compatibilitat amb el `plot_embedding` de `embedded_movies` (no el regenerem). El patró `LangGraph` (`StateGraph`, `MongoDBSaver`) és **agnòstic del proveïdor d'embeddings** — només canvia la funció `generate_embedding()` per cridar `OpenAI` en lloc de `Voyage`.

0.6 Patrons d'arquitectura — workflow vs agent

REGLA: *workflows first, agents only when needed.* Un workflow determinista és sempre més fiable que un agent autònom.

Patró	Quan usar-lo	On al repte
Chaining	passos coneguts, ordre fix	N2: query → retrieve → generate
Routing	tipus de pregunta diferents	N3: classifica → tool adequada
Evaluator-Optimizer	producció + revisió + reintent	N4: context insuficient → re-retrieve
Parallelization	sub-tasques independents	N4 opcional: cercar N gèneres en paral·lel



 **Building with the Claude API** — Anthropic Academy (*secció Agents & Workflows*)

0.7 Tancament ètic — el framework 4D

Quan construeixes un sistema que decideix què respondre a un usuari, t'has de fer 4 preguntes (Anthropic AI Fluency, llicència CC BY-NC-SA):

- **Delegation** — ¿què fa la IA i què feu vosaltres? (cercar a Mongo = mecànic; decidir el to de la resposta = humà).
- **Description** — el prompt és prou específic? (el prompt defensiu és el cas).
- **Discernment** — els documents recuperats són **realment** rellevants? (`score_threshold` no és cosmètic).
- **Diligence** — pots **explicar** per què el RAG va triar aquella resposta?

Riscos a tenir en mente: **al·lucinació** (mitigada amb RAG + prompt defensiu + citar fonts), **biaix del model d'embeddings** (reflecteix el seu entrenament), i **transparència** (l'usuari ha de saber que la resposta està generada).

 **AI Fluency Foundations** — Anthropic Academy

0.8 Recursos per profunditzar (opcionals, gratuïts)

Curs	Badge	Quan
<u>Vector Search Fundamentals</u>	Credly	Durant el lab
<u>RAG with MongoDB</u>	Credly	Per a N2
<u>AI Agents with MongoDB</u>	Credly	Per a N3-N4
<u>AI Data Strategy with MongoDB</u>	Credly	Lectura estratègica
<u>AI Fluency Foundations</u>	Anthropic	Tancament ètic

1. CONTEXT — CINEMATCH

Continuem el fil conductor **CineMatch** (S1-S4). Fins ara heu fet CRUD, agregacions, índexs i disseny sobre `sample_mflix`. Ara construïreu un **assistent intel·ligent** que respon preguntes en llenguatge natural sobre el catàleg de pel·lícules, usant **cerca semàntica** (no `find()` per text exacte) i **generació augmentada per recuperació (RAG)**.

L'objectiu no és “fer servir una IA”, és **entendre l'arquitectura**: com un embedding converteix significat en un vector, com Atlas troba veïns propers, i com es munta un pipeline retriever → generator que no al·lucina.

2. EL DATASET, LA VOSTRA CÒPIA, I UN AVÍS DE CORRECCIÓ CRÍTIC

2.1 El corpus original

`sample_mflix.embedded_movies` és una col·lecció (~3 500 documents) on cada pel·lícula ja porta un camp `plot_embedding` : un vector de **1536 dimensions** calculat prèviament amb el model d'OpenAI `text-embedding-ada-002`.

2.2 Treballem sobre la vostra còpia (no sobre el corpus compartit)

Igual que a la PAUL de S4, **cada grup copia el corpus a la seva base de dades pròpia** (`cinematch_<NIU_lider>`) i hi treballa allà. Així podeu crear índexs, esborrar i experimentar sense afectar la resta de grups. La còpia es fa al minut ~5 del taller:

```
// mongosh – un sol membre del grup ho fa
use cinematch_<NIU_lider>           // p.ex. cinematch_1703702
db.embedded_movies.drop()           // per si l'havies fet abans

// Cal usar $merge (no $out): Atlas no permet $out cross-database
// per a usuaris no-admin. $merge té el mateix efecte.
db.getSiblingDB("sample_mflix").embedded_movies.aggregate([
  { $merge: {
    into: { db: "cinematch_<NIU_lider>", coll: "embedded_movies" },
    on: "_id",
    whenMatched: "replace",
    whenNotMatched: "insert"
  } }
])
db.embedded_movies.countDocuments() // ha de donar ~3 500
```

Lavors creeu el **vostre** índex vectorial sobre la **vostra** col·lecció (no sobre `sample_mflix.embedded_movies`, que és compartit i de només-lectura per a vosaltres):

```
db.embedded_movies.createSearchIndex(
  "vectorPlotIndex", "vectorSearch",
  { fields: [
    { type: "vector", path: "plot_embedding",
      numDimensions: 1536, similarity: "cosine" },
    { type: "filter", path: "year" } ]})
db.embedded_movies.getSearchIndexes() // ull: el camp queryable trigarà 2-3 min
// en passar a true (no pas 1 min). Esperar
// fins queryable:true ABANS d'executar N1.
```

A la plantilla, configureu `MONGODB_URI`, `DB_NAME=cinematch_<NIU_lider>` i `COLLECTION_NAME=embedded_movies` al `.env` (la plantilla ja ho preveu).

2.3 El model d'embedding ha de coincidir EXACTAMENT amb el del corpus

`plot_embedding` està calculat amb `text-embedding-ada-002`. La vostra *query* **ha d'usar el mateix model**. No n'hi ha prou amb que coincideixi la **dimensió** (1536): `text-embedding-3-small` també té 1536 dimensions però viu en un **espai semàntic diferent** → distàncies sense sentit, resultats com soroll. Aquest és un dels **conceptes avaluable**s del Tema 3 (pregunta segura a la defensa).

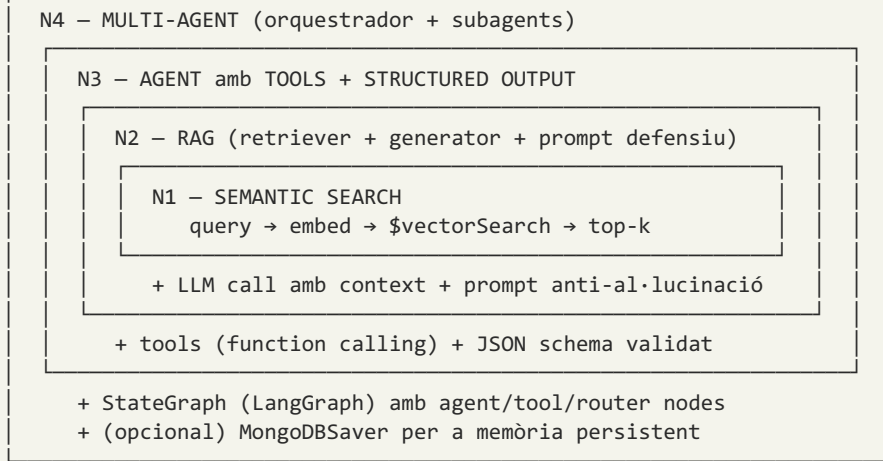
A la plantilla, `EMBED_MODEL` per defecte = `text-embedding-ada-002`. **NO el canvieu** tret que regeneu tot el corpus (fora de l'abast de PLAB1).

⚠ **Nota operativa (pla B docent)**. Si OpenAI retira `ada-002` (és *legacy*), el professor us pot donar una col·lecció pre-regenerada amb `text-embedding-3-small` i els valors corresponents d' `EMBED_MODEL` / `COLLECTION_NAME`. El concepte (model query = model corpus) **NO** canvia. **NO HAURIA DE PASSAR durant el desenvolupament d'aquestes pràctiques**.

3. EL REPTE PER NIVELLS (N1 → N4)

Cada nivell **engloba** l'anterior. La nota depèn del nivell màxim assolit i **funcionant** (veure `rubrica_niveles.md`). És millor un N2 sòlid que un N4 trencat.

Visualització de la progressió arquitectònica:



↑

Cada caixa ****conté**** la d'abans. Trencar la caixa interior trenca totes les exteriors. ⇒ ****build inside-out****.

🌀 N1 — Semantic Search (base, obligatori)

Construïu una funció que, donada una pregunta en llenguatge natural (p. ex. “*space movies where humanity is in danger*”), retorni les **5 pel·lícules més similars** del catàleg.

Heu de: 1. Crear l'índex `vectorSearch` sobre `embedded_movies.plot_embedding` (1536 dims, `cosine`, amb camp `filter` sobre `year`). 2. Embeure la query amb OpenAI (`text-embedding-ada-002`). 3. Executar un pipeline d'agregació amb l'etapa `$vectorSearch` (`numCandidates` , `limit`). 4. Imprimir títol, any i `vectorSearchScore` de cada resultat.

📦 **Lliurable N1:** `n1_semantic_search.py` (o cel·la del notebook) + 3 queries d'exemple amb la seva sortida.

🌀 N2 — RAG (retriever + generator)

A partir de N1, munteu un **pipeline RAG**: la pregunta de l'usuari recupera el context (top-k plots) i un LLM (OpenAI Chat) **redacta una resposta basada NOMÉS en aquest context**.

Heu de: 1. Recuperar top-k documents amb `$vectorSearch` ($k=3..5$). 2. Construir un prompt augmentat: pregunta + plots recuperats + instrucció defensiva (*"if the context doesn't contain the answer, say you don't know; do not invent movies"*). 3. Cridar OpenAI Chat i retornar la resposta **citant** els títols usats. 4. Demostrar amb una pregunta que **sí** es pot respondre amb el context i una que **NO** (ha de dir que no ho sap, no al·lucinar).

📦 **Lliurable N2:** `n2_rag.py` + 2 demos (cas positiu / cas "no ho sé").

🌀 N3 — Tools + Structured Outputs

Converteix el RAG en un **agent amb eines**. L'LLM decideix quina eina cridar i retorna sortida **estructurada** (JSON validable).

Heu de: 1. Definir ≥ 2 *tools* (OpenAI function calling), per exemple: - `search_movies(query, limit)` → `$vectorSearch` - `filter_by_year(min_year, max_year)` o `get_movie_details(title)` 2. Implementar el **loop multi-torn**: model → tool_call → execució → resultat → model → resposta final. 3. La resposta final ha de ser **JSON estructurat** (p. ex.

`{ "recommendations": [ {title, year, why} ], "answered": true }`), validat amb un esquema (Pydantic o `$jsonSchema`).

📦 **Lliurable N3:** `n3_agent_tools.py` + esquema de sortida + 3 casos que exerciten rutes diferents (cerca / filtre / “no trobat”).

🌀 N4 — Multi-agent (orquestració)

Descomponeu el problema en **agents especialitzats** coordinats per un orquestrador.

Heu de: 1. Implementar ≥ 2 subagents amb rol clar, p. ex.: - **Retriever agent** — només cerca/recupera (vector search + filtres). - **Critic/Synthesizer agent** — avalua si el context és suficient i redacta; si no, demana una nova cerca (patró *evaluator-optimizer*). 2. Un **orquestrador** (pot ser LangGraph o un loop propi documentat) que enruti entre subagents segons l'estat. 3. Mostrar un cas on l'orquestrador **reintenta** la recuperació perquè el primer context era insuficient.

📦 **Lliurable N4:** `n4_multiagent.py` + un diagrama (ASCII o imatge) del flux entre agents + traça d'una execució amb reintent.

4. PATRONS D'ARQUITECTURA RECOMANATS

Veure §0.6 per al detall. Resum d'on aplicar-los al repte:

Patró	On l'apliqueu
Chaining (pas → pas → pas)	N2: query → retrieve → generate
Routing (classifica → pipeline)	N3: tipus de pregunta → tool adequada
Evaluator-Optimizer (produir → avaluar → refer)	N4: context insuficient → re-retrieve
Parallelization (split → agg.)	N4 opcional: cercar per N gèneres en paral·lel

REGLA: *workflows first, agents only when needed. Resol-ho de forma fiable abans d'afegir autonomia. Un N2 determinista i correcte val més que un N4 imprevisible.*

5. LLIURABLES I FORMAT

Repositori GitHub **privat** clonat de la **plantilla pública**:

🔗 [ats-2026-plab-tema3](#) — plantilla GitHub (template repo, “Use this template”)

Cliqueu “**Use this template**” → “**Create a new repository**” a GitHub i creeu el vostre repositori amb el nom `<NIU_lider>-cinematch-rag` marcat com a **Private**.

Afegir el professor com a col·laborador: GitHub user `albertgilopez`.

Estructura obligatòria del repositori:

```
<NIU_lider>-cinematch-rag/
├── README.md           ← com executar + nivell assolit + decisions
├── .env.example        ← (MAI pujar .env real ni claus)
├── pyproject.toml
├── src/ o notebook.ipynb
│   ├── n1_semantic_search.py
│   ├── n2_rag.py
│   ├── n3_agent_tools.py   (si arribeu)
│   └── n4_multiagent.py    (si arribeu)
├── docs/
│   ├── DECISIONS.md       ← per què aquest chunking/k/model; trade-offs
│   ├── demo.md           ← captures/traça de cada nivell que entregueu
│   └── IA_REPORT.md       ← registre d'ús d'IA (veure §6)
└── (N4) diagrama del flux d'agents (PNG o ASCII)
```

🔴 **Estructura obligatòria. NO S'AVALUARAN lliuraments que NO segueixin aquesta estructura exacta** (mateixos noms de fitxer, mateixa jerarquia de carpetes). Abans d'entregar, executeu el **validador d'estructura** que es publicarà al canal `#pràctiques-s1` (Discord): comprova automàticament que el vostre repositori té tots els fitxers necessaris en la jerarquia correcta. Si el validador falla, **corregiu-ho abans del lliurament** — **NO** s'acceptaran estructures malformades.

Informe d'ús d'IA obligatori a `docs/IA_REPORT.md` (veure política a §6).

Dates:

Fita	Data
PLAB1 — plantejament + checklist inicial + teoria/investigació (2 h)	21 maig 2026
Tutoria virtual opcional (Teams, a concertar)	22 maig – 3 juny
PLAB2 — sessió de treball (continuació, 2 h) — sense presentacions	4 juny 2026
Lliurament repositori (Campus Virtual)	5 juny 2026, 08:30 h

6. RESTRICCIONS I REGLES

- **Stack fix:** Python, PyMongo, OpenAI, Atlas Vector Search. LangChain/LangGraph **només** permès a N4 si simplifica l'orquestració (a N1-N3 es vol veure el pipeline `$vectorSearch` literal, no amagat per un wrapper).
- **Clau OpenAI:** la proporciona el professor (canal segur). **MAI** committejar la clau ni el `.env`. Un commit amb clau = penalització i rotació immediata.
- **Base de dades:** trebal·leu **SEMPRE** sobre la vostra `cinematch_<NIU_lider>` (teniu credencials `s_<NIU>` amb `readWrite` allà). **NO** intenteu escriure a `sample_mflix.embedded_movies` (lectura sola, és compartit del cluster del curs).
- **Cost per a l'alumne: 0 €.** La clau d'OpenAI la posa el professor; el corpus `embedded_movies` ja porta els embeddings → no cal regenerar res.

🌀 **POLÍTICA D'IA — REGLA.** L'única IA permesa per al desenvolupament d'aquesta pràctica és el bot **PROFE** del curs (Discord). **NO** està permès usar cap altra IA (Claude Code, Claude Copilot, ChatGPT, GitHub Copilot, etc.). **NO S'ACCEPTA lliurar codi que NO sapiguen explicar.**

⚠️ **Auditoria automàtica del cluster.** El cluster del curs (`ats-tema3-sandbox`) té el **profiler de MongoDB actiu**: tota l'activitat que executeu contra la vostra `cinematch_<NIU>` (queries `$vectorSearch`, paràmetres `numCandidates` / `limit`, creació d'índex, timestamps, durada) queda registrada al `system.profile`. Aquesta traça **forma part de l'avaluació**: un repositori amb tests verds però **sense activitat real al cluster** es considera no executat. **NO** intenteu falsificar resultats — l'evidència del profiler ho detecta automàticament.

7. AVALUACIÓ

Veure `rubrica_niveles.md` per al desglossament definitiu. La nota es construeix combinant **6 canals d'evidència** complementaris:

Canal	Què mesura	Com es comprova
Funcionalitat	Cada nivell N1→N4 funciona end-to-end	Tests automàtics del professor sobre el repositori
Qualitat tècnica	Codi reproducible, estructura correcta, MAI secrets committejats	Validador automàtic + static analysis (ruff, gitleaks)
Decisions documentades	<code>DECISIONS.md</code> justifica chunking, <code>k</code> , <code>numCandidates</code> , model, patró	Lectura amb rúbrica
Defensa escrita	5 preguntes obligatòries a <code>docs/demo.md</code> (pre-publicades)	Lectura amb rúbrica estricta
Evidència d'execució	Activitat real al cluster: queries, paràmetres, timestamps	Anàlisi automàtic del <code>system.profile</code> per NIU (veure §6, auditoria automàtica)
Transparència IA	<code>IA_REPORT.md</code> honest sobre ús del bot PROFE	Lectura profe

IMPORTANT: un repositori amb tests verds però **sense activitat real al cluster** (profile buit per al teu NIU) **NO** es considera executat. Els 6 canals es contrasten entre si: incoherències (e.g. claim “N4 funcional” + zero `$vectorSearch` ops al profile) es marquen automàticament per revisió.

8. PER ON COMENÇAR (CHECKLIST MINUT 0)

1. Un membre del grup: crear repo des de la **plantilla**, afegir la resta + professor.
2. `cp .env.example .env` i omplir `MONGODB_URI`, `OPENAI_API_KEY` (del professor) i `DB_NAME=cinematch_<NIU_lider>` (substitueix `<NIU_lider>` pel NIU d'un membre).
3. **Copiar el corpus a la vostra BD** amb mongosh (1 sol cop, ~10s — veure §2.2 per la versió completa amb `$merge`):

```
db.getSiblingDB("sample_mflix").embedded_movies.aggregate([
  { $merge: { into: { db: "cinematch_<NIU_lider>", coll: "embedded_movies" },
              on: "_id", whenMatched: "replace", whenNotMatched: "insert" } }
])
```

Confirmar amb `db.embedded_movies.countDocuments()` → ~3500.

4. `uv sync` (o verificar si ja teniu l'entorn).
5. **Crear l'índex vectorial** sobre la **vostra** col·lecció (no sobre `sample_mflix`):
`db.embedded_movies.createSearchIndex(...)` segons §2.2. Esperar `READY` (~1 min).
6. Executar `uv run python -m src.n1_semantic_search "test"` → ha de retornar 5 pel·lícules. Si funciona, ja teniu N1 base; a partir d'aquí, construïu.

Mini-Agentic-RAG sobre CineMatch

Pràctica de Laboratori — Tema 3
NoSQL (MongoDB) + IA Generativa

AUTOR

Albert Gil López
albert.gil.lopez@uab.cat
github.com/albertgilopez

CALENDARI

PLAB1 · 21 maig 2026 (2 h)
PLAB2 · 4 juny 2026 (2 h)
Lliurament · 5 juny 2026, 08:30 h